Intro to Data Science - Lab 7

Copyright 2025, Jeffrey Stanton and Jeffrey Saltz Please do not post online.

Week 7 - Using ggplot to Build Complex Data Displays

```
# Enter your name here: Tim Soron
```

Please include nice comments.

Instructions:

Run the necessary code on your own instance of R-Studio.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this lab assignment by myself, with help from the book and the professor.
```

Geology rocks but geography is where it's at. . . (famous dad joke). In a global economy, geography has an important influence on everything from manufacturing to marketing to transportation. As a result, most data scientists will have to work with map data at some point in their careers.

An add-on to the **ggplot2** package, called **ggmap**, provides powerful tools for plotting and shading maps. Make sure to install the **maps**, **mapproj**, and **ggmap** packages before running the following:

```
#install.packages("maps")
#install.packages("ggmap")
#install.packages("mapproj")
```

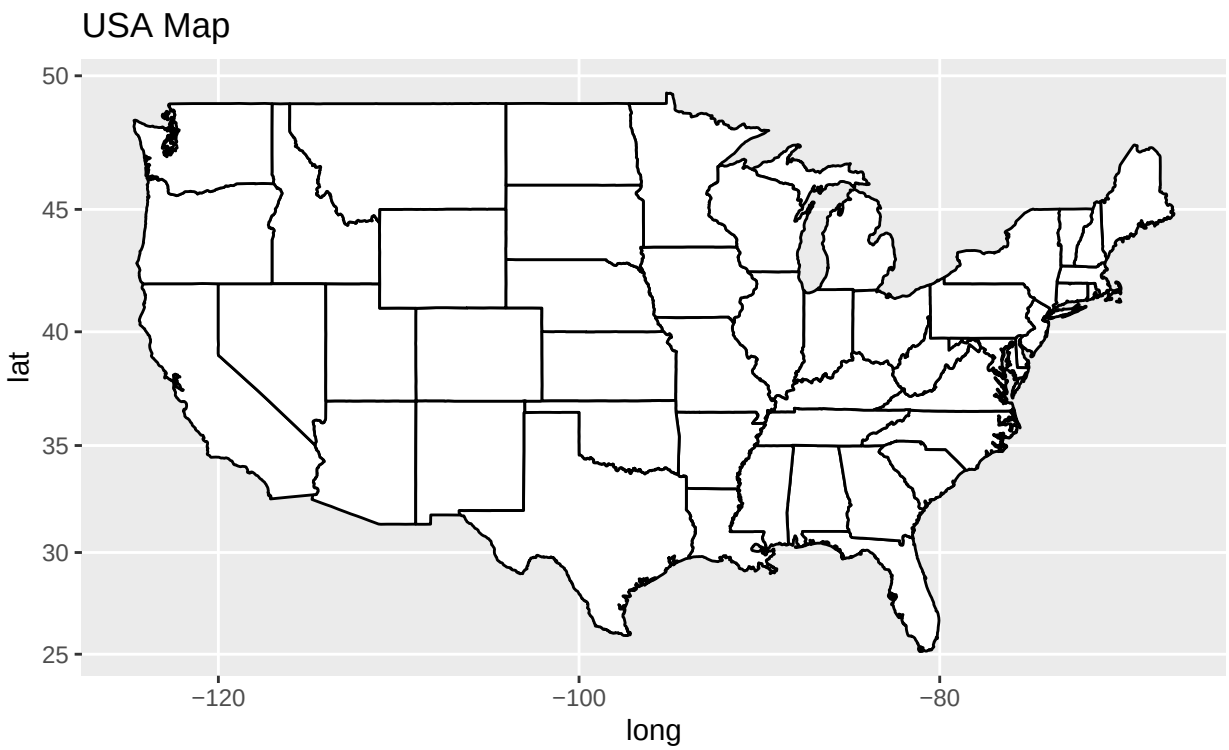
```
library(ggplot2); library(maps); library(ggmap); library(mapproj)
us <- map_data("state")
us$state_name <- tolower(us$region)
map <- ggplot(us, aes(map_id= state_name))
map <- map + aes(x=long, y=lat, group=group) +
geom_polygon(fill = "white", color = "black")
map <- map + expand_limits(x=us$long, y=us$lat)
map <- map + coord_map() + ggtitle("USA Map")
map
```

1. Paste the code below and add a comment for each line, explaining what that line of code does.

```
library(ggplot2); library(maps); library(ggmap); library(mapproj)
```

```
## i Google's Terms of Service: <https://mapsplatform.google.com>
##   Stadia Maps' Terms of Service: <https://stadiamaps.com/terms-of-service>
##   OpenStreetMap's Tile Usage Policy: <https://operations.osmfoundation.org/policies/tiles>
## i Please cite ggmap if you use it! Use `citation("ggmap")` for details.
```

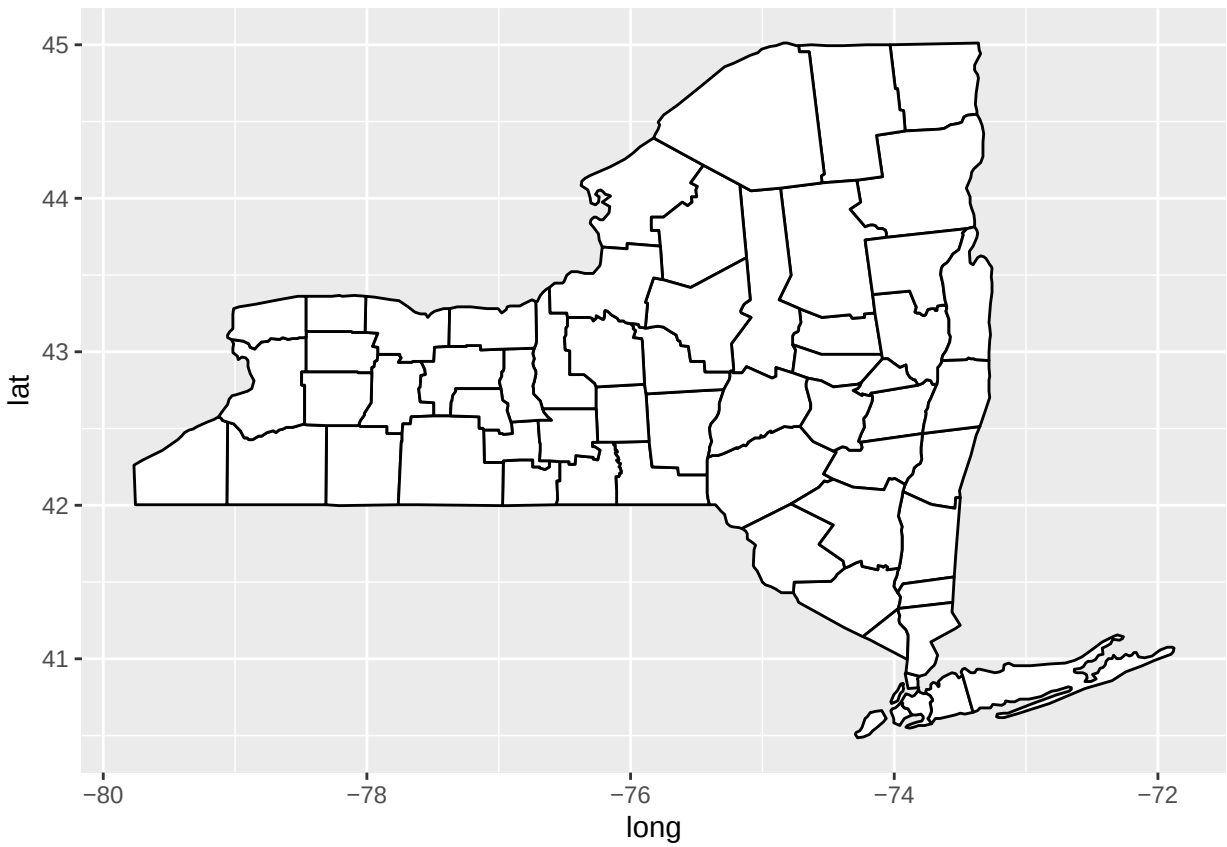
```
us <- map_data("state") # returns data frame with state values
us$state_name <- tolower(us$region) # changes region column to state name
ggplot(us, aes(x=long, y=lat, group=group))+
  geom_polygon(fill="white", color="black")+
  expand_limits(x=us$long, y=us$lat)+
  coord_map() + ggtitle("USA Map")
```



2. The map you just created fills in the area of each state in white while outlining it with a thin black line. Use the **fill=** and **color=** commands inside the call to **geom_polygon( )** to reverse the color scheme. Now paste and run the following code:

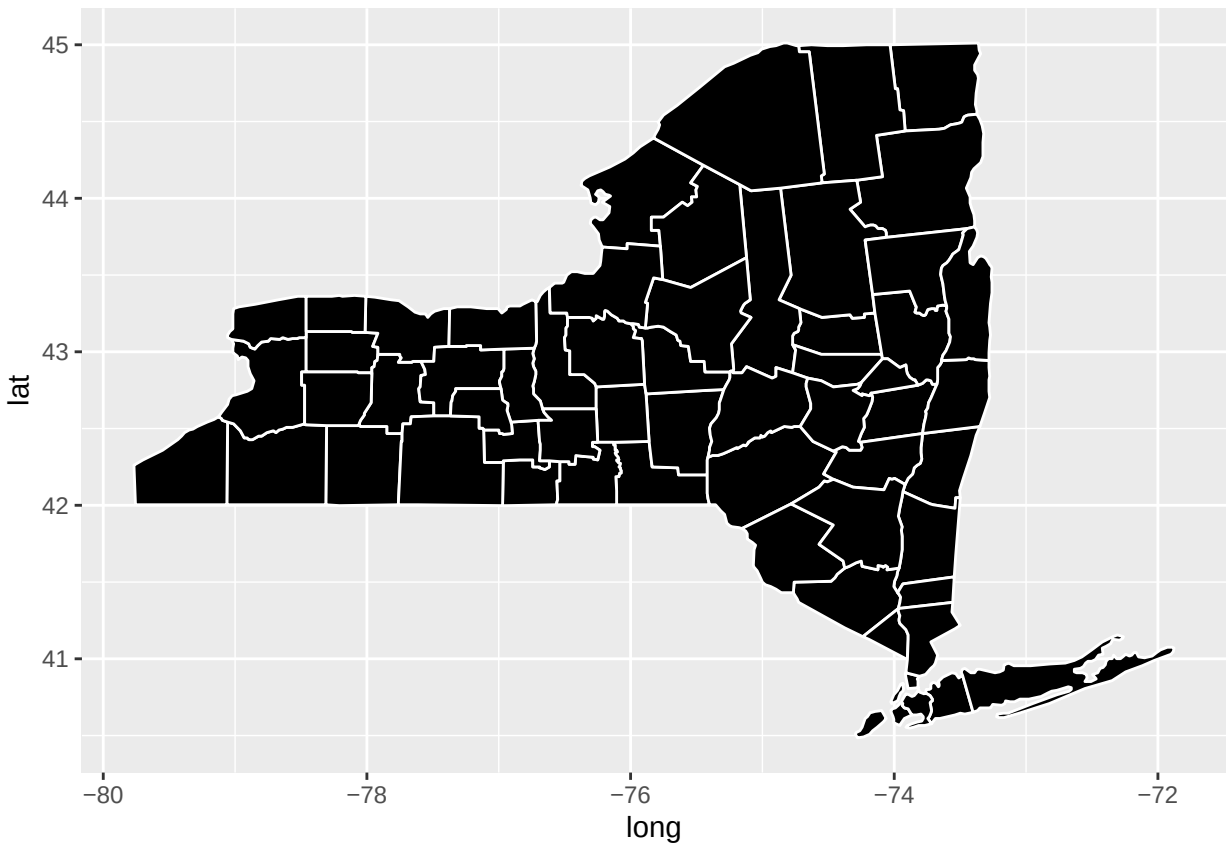
```
ny_counties <- map_data("county","new york")
ggplot(ny_counties) + aes(long,lat, group=group) + geom_polygon(fill
= "white", color = "black")
```

```
ny_counties <- map_data("county","new york")
ggplot(ny_counties) + aes(long,lat, group=group) + geom_polygon(fill
= "white", color = "black")
```



3. Just as in step 2, the map you just created fills in the area of each county in black while outlining it with a thin white lines. Use the **fill=** and **color=** commands inside the call to **geom_polygon( )** to reverse the color scheme.

```
ggplot(ny_counties) + aes(long,lat, group=group) + geom_polygon(fill
= "black", color = "white")
```



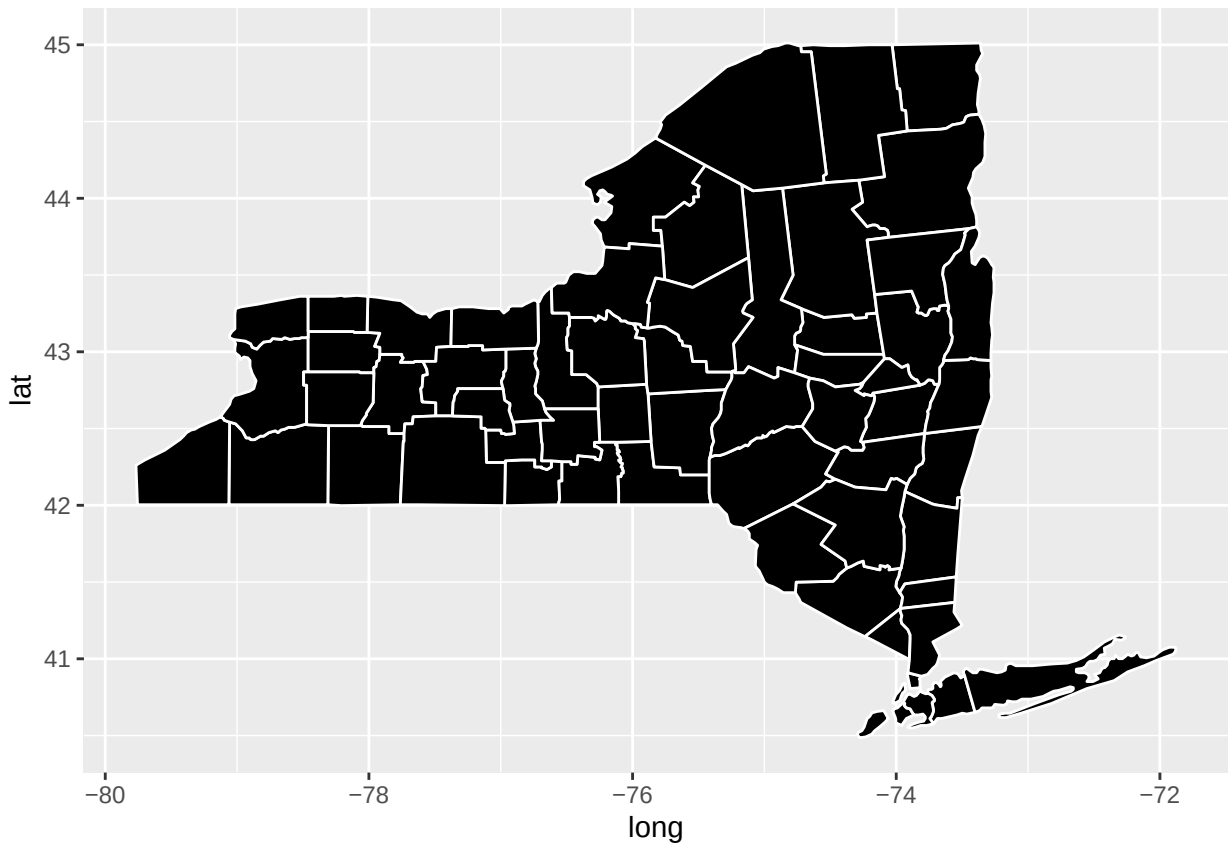
4. Run `head(ny_counties)` to verify how the county outline data looks

```
head(ny_counties)
```

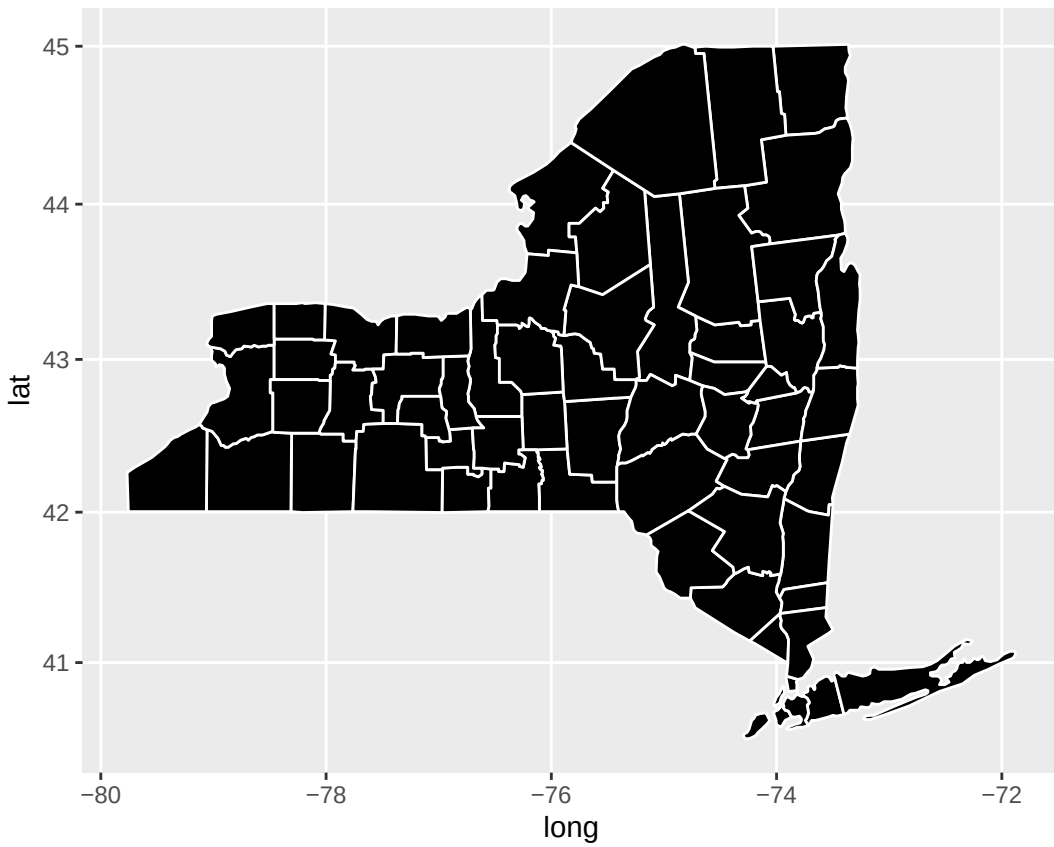
```
##        long      lat group order   region subregion
## 1 -73.78550 42.46763     1     1 new york    albany
## 2 -74.25533 42.41034     1     2 new york    albany
## 3 -74.25533 42.41034     1     3 new york    albany
## 4 -74.27252 42.41607     1     4 new york    albany
## 5 -74.24960 42.46763     1     5 new york    albany
## 6 -74.22668 42.50774     1     6 new york    albany
```

5. Make a copy of your code from step 3 and add the following subcommand to your ggplot( ) call (don t forget to put a plus sign after the **geom_polygon( )** statement to tell R that you are continuing to build the command): `coord_map(projection = "mercator")` In what way is the map different from the previous map. Be prepared to explain what a Mercator projection is.

```
ggplot(ny_counties) + aes(long,lat, group=group) + geom_polygon(fill
= "black", color = "white")
```




```
ggplot(ny_counties) + aes(long,lat, group=group) + geom_polygon(fill
= "black", color = "white")+
  coord_map(projection = "mercator")
```



6. Grab a copy of the nyData.csv data set from: https://intro-datascience.s3.us-east-2.amazonaws.com/nyData.csv Read that data set into R with **read_csv()**. This will require you have installed and libraried the **tidyverse** package. The next step assumes that you have named the resulting data frame ** nyData. **

```
install.packages("tidyverse")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'
## (as 'lib' is unspecified)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v lubridate 1.9.5     v tibble    3.3.1
## v purrr     1.2.1     v tidyr     1.3.2
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## x purrr::map()    masks maps::map()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
nyData <- read_csv("https://intro-datascience.s3.us-east-2.amazonaws.com/nyData.csv")
```

```
## Rows: 62 Columns: 5
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): county
```

```
## num (4): pop2010, pop2000, sqMiles, popDen
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

7. Next, merge your **ny_counties** data from the first set of questions with your new **nyData** data frame, with this code: `mergeNY <- merge(ny_counties,nyData,all.x=TRUE,by.x="subregion",by.y="county")`

```r
mergeNY <- merge(ny_counties,nyData,all.x=TRUE,by.x="subregion",by.y="county")
```

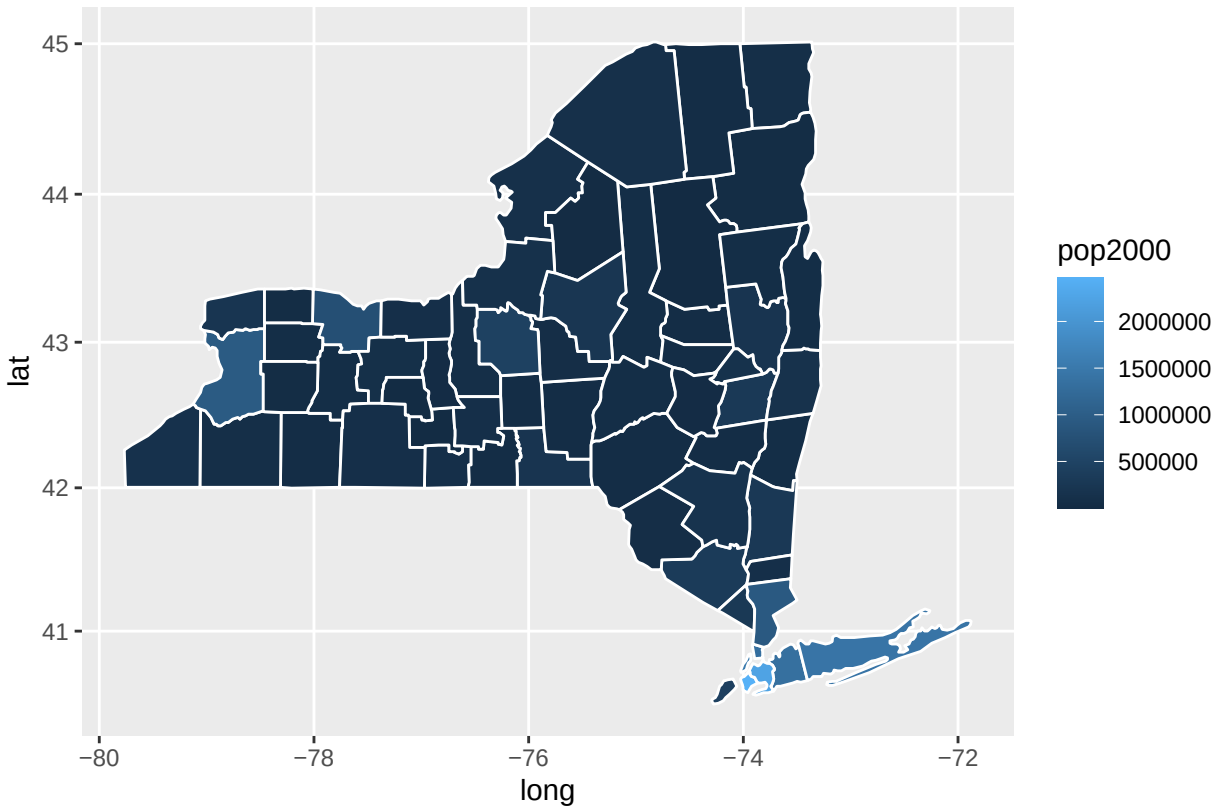
8. Run `head(mergeNY)` to verify how the merged data looks.

```r
head(mergeNY)
```

```
##   subregion      long      lat group order   region pop2010 pop2000 sqMiles
## 1    albany -73.78550 42.46763     1     1 new york  304204  294565   522.8
## 2    albany -74.25533 42.41034     1     2 new york  304204  294565   522.8
## 3    albany -74.25533 42.41034     1     3 new york  304204  294565   522.8
## 4    albany -74.27252 42.41607     1     4 new york  304204  294565   522.8
## 5    albany -74.24960 42.46763     1     5 new york  304204  294565   522.8
## 6    albany -74.22668 42.50774     1     6 new york  304204  294565   522.8
##   popDen
## 1 581.87
## 2 581.87
## 3 581.87
## 4 581.87
## 5 581.87
## 6 581.87
```

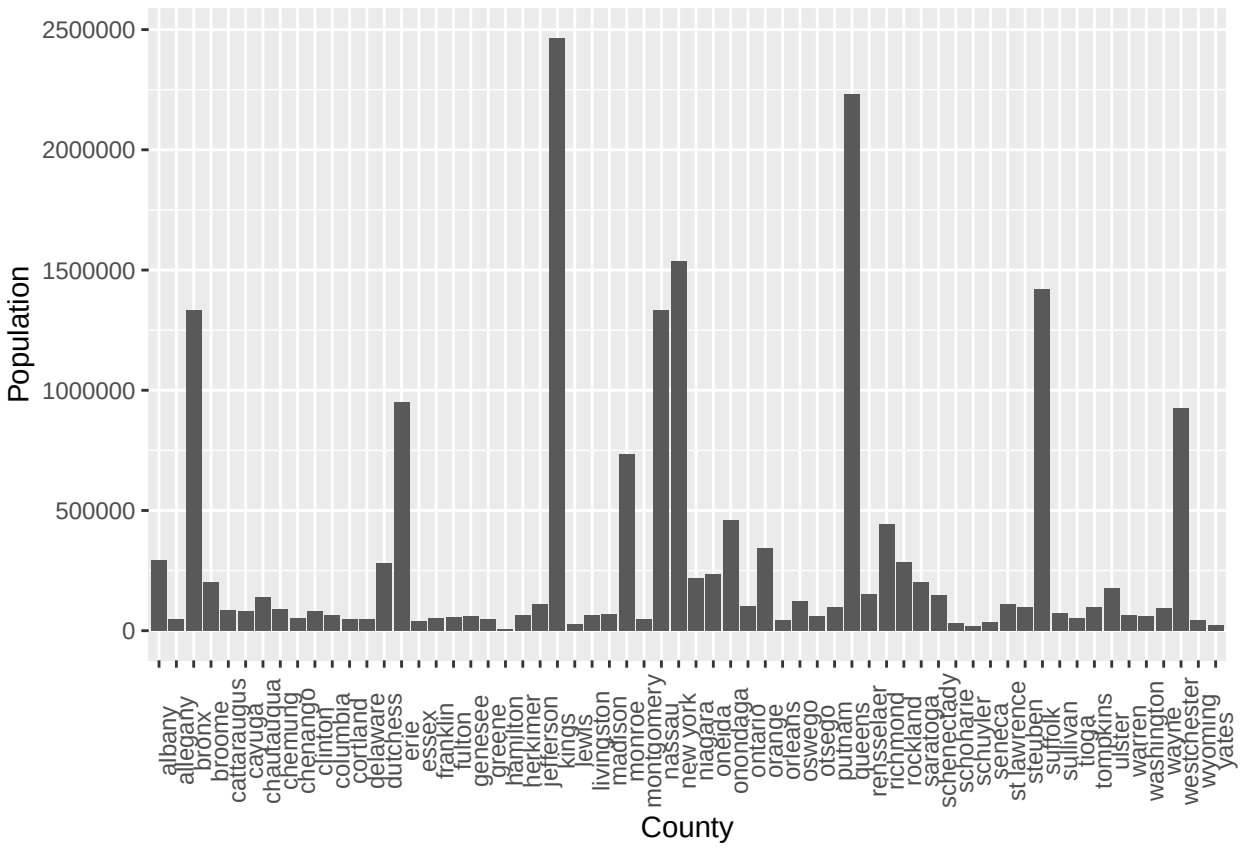
9. Now drive the fill color inside each county by adding the **fill** aesthetic inside of your **geom_polygon( )** subcommand (fill based on **pop2000**).

```r
ggplot(mergeNY)+
  aes(long,lat, group=group)+
  geom_polygon(color = "white", aes(fill=pop2000))+
  coord_map(projection = "mercator")
```



10. Create a barchart using ggplot (each county is a bar, the height should be based on **pop2000**)

```
ggplot(data=nyData, aes(x=county, y=pop2000))+
  geom_col()+
  theme(axis.text.x=element_text(angle=90))+
  xlab("County")+
  ylab("Population")
```



11. In a comment, compare the visualization in 9 & 10. Is one easier to understand (you must explain why).

```
# I think that 10 is easier to understand if you wanted to see more exact values for the counties, but
```